

Relatório de Auditoria de Segurança — Plannex

Data: 10/10/2026 · Repositório GotardoN1/Plannex, branch master, commit 36182b0

ESCOPO AUDITADO

Todo o backend (src/index.js, src/arquivos.js, src/senha.js, src/demo.js), o frontend da Central (public/painel/js), o JavaScript e o HTML do site público, public/_headers, wrangler.jsonc, as 16 migrações, tools/, tests/, README e os 49 commits do histórico git. O código foi lido por inteiro e os controles de acesso foram exercitados contra o próprio Worker, num banco em memória com dados fictícios.

NOTA METODOLÓGICA — COMO CADA CATEGORIA FOI MAPEADA PARA A STACK

C1 · Banco sem tranca (isolamento). Projeto de inquilino único (uma empresa). O isolamento é por papel e por dono: o funcionário só pode alcançar contatos com responsavel_id = ele, fora de arquivados, recusados e da etapa do administrador. Mecanismo: contatoVisivel() e filtros SQL em central() e consultaAtividade(). Também foi verificado o isolamento entre solicitantes anônimos do formulário público.

C2 · Permissão definida no navegador. Cada gate de papel do frontend (TELAS admin em app.js, eAdmin() em ficha.js, materiais.js e demais) foi cruzado com a rota da API que ele aciona, conferindo a checagem no servidor.

C3 · IDOR. Todas as rotas da função api() (src/index.js:179-279) foram listadas e cada handler que recebe ID foi seguido até a consulta, conferindo a verificação de posse ou de papel.

C4 · Chaves expostas (hardcode). Varredura da árvore atual, dos 49 commits do histórico git, de wrangler.jsonc, ferramentas, testes, README e do JavaScript entregue ao navegador. Não há variáveis de ambiente secretas nem defaults do tipo \${VAR:-valor}; a sessão não depende de chave de assinatura.

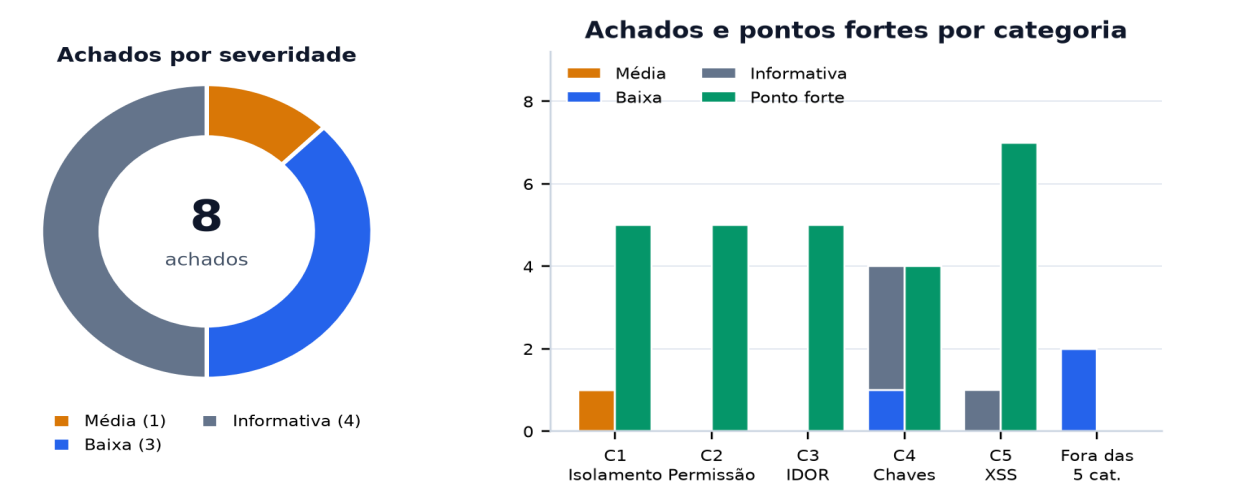
C5 · Inputs sem tratamento (XSS). Busca de sinks (innerHTML, insertAdjacentHTML, eval, new Function, href/src com dado do usuário, location.*) e de fontes (hash, query, postMessage). Backend: respostas HTML, cabeçalhos com nomes de arquivo, CSV e PDF gerados. Não há lib de sanitização: o projeto evita HTML por construção.

Só entram achados verificados no código real. Os itens marcados como reproduzidos foram confirmados executando o Worker. Achados fora das cinco categorias aparecem à parte e não alteram a análise delas.

1. Resumo executivo

0	0	1	3	4	26
Crítica	Alta	Média	Baixa	Informativa	Pontos fortes

A Central está bem protegida nos pontos que mais importam: **não foi encontrada nenhuma falha crítica ou alta**, nenhum IDOR, nenhuma permissão decidida só no navegador e nenhum segredo no código ou no histórico. O isolamento entre administrador e funcionário é aplicado no servidor em todas as rotas, e o frontend não usa HTML com dados de usuário, com uma CSP rígida por cima. O risco central está no **formulário público**: a regra anti-duplicidade faz pedidos de pessoas diferentes com o mesmo nome sumirem sem rastro (média). Os demais itens são proteções contra abuso com brechas pequenas e registros informativos.



Total: 8 achados (6 nas cinco categorias e 2 fora delas). Os informativos não pedem correção; viram issue só os acionáveis.

Stack detectada

Camada	O que foi encontrado
Linguagem	JavaScript (ES modules), sem TypeScript nem build
Runtime / framework	Cloudflare Workers, roteamento manual em src/index.js (sem framework)
Banco / acesso	Cloudflare D1 (SQLite) com prepared statements e bind(); sem ORM
Arquivos	Cloudflare KV (binding ARQUIVOS); metadados na tabela arquivos
Autenticação	Usuário e senha (PBKDF2-SHA256, 100 mil iterações) e sessão em cookie HttpOnly, Secure, SameSite=Strict; token aleatório de 32 bytes guardado como SHA-256 na tabela sessoes
Autorização	Dois papéis (admin, funcionario) e posse por contatos.responsavel_id
Frontend	Site estático (public/) e Central em JS puro (public/painel/js), DOM montado com textContent
Deploy / CI	wrangler.jsonc (produção e ambiente demo); sem Docker, Helm, Terraform ou CI no repositório
Terceiros	FormSubmit (e-mail do formulário, chamado pelo navegador); pdf-lib embarcado (vendedor)

2. Pontos fortes e pontos fracos

Pontos fortes — o que está protegido, com evidência

C1 · Banco sem tranca (isolamento)

- `contatoVisivel()` (`src/index.js:281-286`) é aplicado antes de todo handler de `/api/contatos/:id` (224-237), de arquivos (1175-1179) e de notas (1095).
- `/api/central` filtra os contatos do funcionário no SQL e zera telefone, e-mail, CPF, valor, nota fiscal e pagamento antes de responder (790, 816-824); usuários: só ele mesmo (806).
- Atividade e linha do tempo filtram por responsável e escondem anotações restritas (`consultaAtividade` 762-777; `linhaDoTempo` 1056-1068).
- Arquivos, moldes e materiais filtrados por categoria e visibilidade nas listagens (`listarArquivos` 1115-1123; `central` 800, 809-813).
- Sonda executada contra o Worker: a funcionária Bia recebeu só os contatos 10 e 11, com os campos de contato e financeiros nulos.

C2 · Permissão definida no navegador

- Todas as telas marcadas `admin: true` (`public/painel/js/app.js:24-33`) têm checagem no servidor: financeiro e despesas (`src/index.js:206-216`), equipe (222, 269-276), novo contato (221).
- `alterarContato` aceita do funcionário só as chaves `etapa`, `lido` e `iniciado` (858) e limita as etapas (861-872); a anotação na aba Notas e ordens exige `admin` (1086).
- Categorias de arquivo do administrador (`nota`, `ordem`, `outro`) barradas no envio, listagem e download (1111, 1116, 1134, 1177); molde da OS só para `admin` (246).
- CSRF: toda escrita exige `Origin` igual ao site (184) e o cookie é `SameSite=Strict` (558).
- Modo demonstração (`entrar sem senha`) só liga com três marcas: `DEMO=true`, `AMBIENTE=demo`, `banco` e `KV` marcados como `demo` (86-115). Testado: em produção `/api/demo/entrar` responde 404.

C3 · IDOR

- Todas as 42 combinações de rota e método foram percorridas (ver Cobertura). Nenhum handler busca, altera ou apaga por ID sem conferir papel ou posse.
- `DELETE /api/etiquetas/:id` apaga só com `usuario_id` do chamador (721); `DELETE /api/notas/:id` exige demanda visível e autoria (1091-1098).
- `GET` e `DELETE /api/arquivos/:id` passam por `arquivoVisivel()` (1175-1179), e o funcionário só apaga o que ele enviou e com a demanda aberta (1346-1351).
- `GET /api/materiais/:id` respeita visibilidade `admin` (1263); rotas de usuários e despesas exigem `admin`.
- Sonda: 9 tentativas de IDOR da funcionária contra objetos alheios foram barradas (404/403; na etiqueta alheia, 200 sem apagar nada).

C4 · Chaves expostas (hardcode)

- Nenhum segredo na árvore nem nos 49 commits; nenhum `.env` ou `.dev.vars` versionado (`.gitignore` cobre `.dev.vars` e backups).
- Não existe segredo de assinatura a vazar: a sessão é um token aleatório de 32 bytes guardado como SHA-256 (553-559, 603-610).
- Senhas com PBKDF2-SHA256, sal de 16 bytes e comparação em tempo constante (`src/senha.js:9-33`); login confere um hash falso para não denunciar usuários inexistentes (539-541).
- `tools/criar-usuario.mjs` pede a senha sem ecoar e grava só o hash; as migrações não criam usuário padrão.

C5 · Inputs sem tratamento (XSS)

- A Central monta o DOM com `el()` e `textContent` (`public/painel/js/utl.js:2, 39-47`); o único `innerHTML` recebe ícones fixos (`utl.js:135`).
- No site, `innerHTML` só recebe textos fixos (`script.rev122.js:876, 911, 916, 927`).
- Links de WhatsApp e e-mail têm esquema fixo e `encodeURIComponent` (`mensagens.js:115-124`): sem `javascript:`. Rotas por hash são validadas contra a lista de telas (`app.js:46-49`).
- CSP sem `unsafe-inline` nem `unsafe-eval`, `frame-ancestors 'none'`, `nosniff` e HSTS (`public/_headers:1-10`).
- Downloads sempre como anexo `octet-stream` com `nosniff` (`src/index.js:1187-1195, 1205-1213, 1266-1274`); nomes de arquivo higienizados (1154) e conteúdo conferido pela assinatura (`src/arquivos.js`).
- Exportação CSV neutraliza fórmulas `=`, `+`, `-` e `@` (`entrada.js:280-283`) e o servidor remove tabulação e CR dos textos (`src/index.js:1418-1426`).
- SQL dinâmico só com nomes de coluna fixos no código (434-449, 471-480, 292-303); valores sempre por `bind()`.

Pontos fracos — os riscos centrais

- O único achado de severidade média está no formulário público: a regra anti-duplicidade usa o nome como identidade, o que faz pedidos de pessoas diferentes colidirem e sumirem sem rastro (A1).
- O canal de e-mail depende de um terceiro configurado com o endereço real e sem captcha, fora do limite de envios da Central (A2, A6).
- Duas proteções contra abuso têm brecha: o bloqueio de login por usuário vira ferramenta para travar contas (A7) e o teto de 32 KB do JSON depende de um cabeçalho que o cliente controla (A8).

3. Achados detalhados por categoria

Arquivo por arquivo, linha por linha. Caminhos relativos à raiz do repositório.

C1 · Banco sem tranca (isolamento)

Severidade	Arquivo:linha	Descrição
MÉDIA	src/index.js:362-367 src/index.js:326-333 public/assets/js/script.rev122.js:889	A1 · Formulário público descarta o pedido de outra pessoa com o mesmo nome e devolve o protocolo dela (issue 1) A deduplicação do POST /api/contato considera repetido qualquer pedido com o mesmo serviço e o mesmo nome nos últimos 10 minutos, sem olhar e-mail, telefone ou texto. A segunda pessoa recebe ok, repetido: true e o protocolo do primeiro pedido; nada é gravado e o navegador também deixa de enviar o e-mail do FormSubmit. Reproduzido com o Worker: duas "Maria Souza" com e-mails, telefones e descrições diferentes resultaram em um único contato (o da primeira).

C2 · Permissão definida no navegador

Nenhum achado. Todas as verificações desta categoria passaram; a evidência está em Pontos fortes e na tabela de Cobertura.

C3 · IDOR

Nenhum achado. Todas as verificações desta categoria passaram; a evidência está em Pontos fortes e na tabela de Cobertura.

C4 · Chaves expostas (hardcode)

Severidade	Arquivo:linha	Descrição
BAIXA	public/index.html:393-396 public/assets/js/script.rev122.js:460	A2 · Endpoint do FormSubmit com o e-mail real e captcha desligado (issue 4) O formulário publica o endereço contato@plannex.online como endpoint do FormSubmit e envia _captcha=false. Esse identificador funciona como credencial de envio: qualquer pessoa pode disparar e-mails para a caixa da Plannex direto no FormSubmit, sem passar pelo limite de 5 envios a cada 10 minutos que a Central aplica.
INFORMATIVA	wrangler.jsonc:22 wrangler.jsonc:28 wrangler.jsonc:53 wrangler.jsonc:58	A3 · IDs do D1 e do KV versionados database_id e id dos namespaces KV estão no repositório público. Não são credenciais (sem um token da conta Cloudflare não dão acesso), é o padrão do wrangler. Registrado só para inventário.
INFORMATIVA	tests/apoio.mjs:9 README.md:147	A4 · Senhas de teste no código 'senha-de-teste-123' e 'uma-senha-de-teste' só existem no banco em memória dos testes e no exemplo do servidor local; nenhum usuário real é criado com elas (as migrações não inserem usuários).
INFORMATIVA	public/painel/js/mensagens.js:6	A5 · Chave PIX embutida no JavaScript A chave PIX vai em toda mensagem de aceite para o cliente; é pública por natureza e não é segredo. Fica só o registro de que é um dado pessoal (número de telefone) servido em /painel/js.

C5 · Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição
INFORMATIVA	public/index.html:395 public/assets/js/script.rev122.js:871 public/assets/js/script.rev122.js:887	A6 · E-mail do formulário montado por terceiro com dados do cliente (issue 4) Nome e campos livres entram no assunto e no corpo do e-mail gerado pelo FormSubmit (template "table"). O escape desse HTML fica fora do controle do projeto; não foi possível verificar.

Fora das cinco categorias

Encontrados durante a leitura completa do código; não pertencem às cinco categorias pedidas, mas são acionáveis.

Severidade	Arquivo:linha	Descrição
BAIXA	src/index.js:533-537 src/index.js:543-546	A7 · Qualquer pessoa trava o login de um usuário por 15 minutos (issue 2) O limite de 5 tentativas por usuário vale para qualquer IP. Sabendo um login (ex.: o do administrador), um anônimo erra 5 vezes e a conta fica bloqueada, mesmo com a senha certa. Reproduzido: login correto da Ana respondeu 429 depois de 5 erros de terceiros.
BAIXA	src/index.js:1413-1416 src/index.js:350	A8 · Limite de 32 KB do JSON depende do Content-Length (issue 3) lerjson() só recusa corpos grandes quando o cabeçalho Content-Length existe. Sem ele (envio em stream/chunked), request.json() lê o corpo inteiro. Reproduzido: 5 MB sem Content-Length foram aceitos em PATCH /api/eu. Vale também para o POST /api/contato, que é público.

Cobertura das rotas (C2 e C3)

Todas as rotas da API em src/index.js e o controle que o servidor aplica em cada uma.

Rota	Controle no servidor	Linhas
POST /api/contato	Público: limite por IP, honeypot, validação de arquivos	335-395
/api/demo/*	404 sem demonstração confirmada	189, 86-115
POST /api/login · POST /api/sair	Limite por IP e usuário; hash falso; apaga a sessão	527-551, 595-601
GET /api/sessao · GET /api/versao	Sessão obrigatória	195-203
GET /api/financeiro · PATCH /api/empresa	Só admin	206-207
POST /api/despesas · PATCH, DELETE /api/despesas/:id · POST .../renovar	Só admin	208-216
GET /api/central · GET /api/atividade	Filtro por papel e responsável	762-826
POST /api/senha · PATCH /api/eu	Senha atual; só o próprio id	612-628, 688-702
POST /api/contatos · POST /api/usuarios	Só admin	221-222
PATCH, DELETE /api/contatos/:id	contatoVisivel; lista de chaves do funcionário; DELETE só admin	228-231, 858
GET /api/contatos/:id/linha-do-tempo	contatoVisivel; esconde restritas	232, 1056-1068
POST /api/contatos/:id/notas	contatoVisivel; aba do admin; demanda fechada	233, 1081-1089

Rota	Controle no servidor	Linhas
GET, POST /api/contatos/:id/arquivos	contatoVisivel; categoria por papel	234-235, 1115-1148
POST /api/contatos/:id/etiquetas	contatoVisivel; etiqueta pessoal	236, 707-718
GET, DELETE /api/arquivos/:id	arquivoVisivel; autor e etapa	239-241, 1175-1196, 1343-1359
GET, POST, DELETE /api/moldes/:tipo	OS só admin; envio e exclusão só admin	243-250
POST /api/materiais · GET, POST, PATCH, DELETE /api/materiais/:id	Visibilidade; escrita só admin	252-261, 1261-1341
DELETE /api/etiquetas/:id	Filtra por usuario_id	263-264, 720-723
DELETE /api/notas/:id	Demanda visível e autoria	266-267, 1091-1098
PATCH, DELETE /api/usuarios/:id · POST .../senha	Só admin; não altera a si mesmo	269-276

4. Recomendações priorizadas

P1**Corrigir a deduplicação do formulário (A1)**

Considerar repetido só o mesmo pedido (e-mail ou telefone e o mesmo texto, ou um hash do conteúdo todo); nunca devolver o protocolo de outro registro; quando houver dúvida, gravar o contato e deixar a triagem para a caixa de entrada. Adicionar teste com duas pessoas de mesmo nome.

P2**Endurecer o canal de e-mail (A2, A6)**

Trocar o endereço pelo alias aleatório do FormSubmit, ligar o captcha ou, melhor, enviar o e-mail pelo próprio Worker (depois do limite por IP) com um serviço transacional e template escapado.

P3**Bloqueio de login sem efeito colateral (A7)**

Contar falhas por usuário e IP juntos, usar espera progressiva em vez de bloqueio total e/ou exigir Cloudflare Turnstile depois de algumas falhas.

P4**Teto de tamanho que não dependa do cliente (A8)**

Ler o corpo como stream com limite de bytes (ou recusar requisição sem Content-Length) antes do request.json(), no formulário público e na API.

P5**Transformar a sonda desta auditoria em testes de regressão**

As tentativas de IDOR e de escalada de papel que foram barradas viram um tests/seguranca.test.mjs, para a proteção não regredir em mudanças futuras.

5. ISSUES PARA O GITHUB

Um bloco por achado acionável, pronto para copiar e colar. A primeira linha de cada bloco é o título e a segunda, as labels sugeridas; o restante é o corpo em Markdown. A6 foi agrupado com A2 (mesmo tema: FormSubmit). A3, A4 e A5 são informativos e não geram issue.

--- ISSUE 1 ---

Título: [Segurança] Formulário público descarta o pedido de outra pessoa com o mesmo nome e devolve o protocolo dela

Labels: security, severidade: média

Descrição

A deduplicação do `POST /api/contato` trata como "repetido" qualquer pedido com o **mesmo serviço e o mesmo nome** recebido nos últimos 10 minutos, sem comparar e-mail, telefone ou texto. Duas pessoas diferentes com nome comum (ex.: "Maria Souza") colidem: a segunda recebe `ok: true`, `repetido: true` e o **protocolo da primeira**, nada é gravado na Central e o navegador também **deixa de enviar o e-mail** do FormSubmit. O pedido some sem rastro.

É explorável de dois jeitos: por acaso (nomes comuns no mesmo serviço) ou de propósito, por quem envia um pedido com o nome de outra pessoa logo antes dela, fazendo o pedido real ser descartado.

Evidência

```
`src/index.js:362-367`
```js
const repetido = await env.DB.prepare(
 "SELECT id, protocolo FROM contatos WHERE servico = ? AND nome = ? AND criado_em > " +
 "strftime('%Y-%m-%dT%H:%M:%SZ', 'now', '-10 minutes')" // (linha 364, quebrada aqui para caber)
).bind(servico, campos.nome).first() || await pedidoRepetido(env, campos);
...
if (repetido) return json({ ok: true, repetido: true, protocolo: repetido.protocolo }, 200);
...
`public/assets/js/script.rev122.js:889`
```js
const emailOk = central.repetido ? true : await fetch(FORM_SUBMIT_ENDPOINT, { ... })
...

```

Reprodução (harness dos testes): duas "Maria Souza" com e-mails, telefones e descrições diferentes -> a segunda recebeu `{"ok":true,"repetido":true,"protocolo":"PLX-2026-0001"}` e o banco ficou com um único contato.

Impacto

Perda silenciosa de pedidos de clientes (sem registro na Central e sem e-mail) e exposição do protocolo de outra pessoa ao solicitante.

Sugestão de correção

- Remover a regra por `servico + nome`; manter só `pedidoRepetido` (e-mail **ou** telefone + mesma descrição), ou comparar um hash de todos os campos enviados.
- Não devolver o protocolo de outro registro: no caso repetido, responder só `ok: true, repetido: true`.
- Na dúvida, gravar o contato e deixar a triagem para a caixa de entrada.

Critérios de aceite

- [] Dois envios com mesmo nome e serviço, mas e-mail/telefone ou texto diferentes, geram dois contatos.
- [] Reenvio idêntico (clique duplo) continua sem duplicar.
- [] A resposta de um envio repetido não traz o protocolo de outro registro.
- [] Teste automatizado cobrindo os três casos acima.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Qualquer pessoa consegue travar o login de um usuário por 15 minutos

Labels: security, severidade: baixa

Descrição

O limite de tentativas de login conta falhas por **usuário**, vindas de qualquer IP. Quem souber um login (ex.: o do administrador) erra a senha 5 vezes e a conta fica bloqueada por 15 minutos, inclusive para o dono com a senha certa. Repetindo a cada 15 minutos, o bloqueio é permanente.

Evidência

```
`src/index.js:533-537`
```js
const chaveIp = `login-ip:${await hashIp(request)}`;
```

```
const chaveUsuario = `login-usuario:${usuario}`;
if (await excedeuLimite(env, chaveIp, LIMITE_LOGIN)
 || await excedeuLimite(env, chaveUsuario, LIMITE_LOGIN)) { // (linha 535, quebrada aqui para
 caber)
 return json({ erro: 'Muitas tentativas. Aguarde 15 minutos.' }, 429);
},
},
},
```

Reprodução: após 5 falhas de terceiros, o login correto da usuária `ana` respondeu `429`.

#### ## Impacto

Negação de acesso à Central para administradores e equipe.

#### ## Sugestão de correção

- Contar falhas por `usuário + IP`; manter o limite por IP como está.
- Para o limite global por usuário, usar espera progressiva ou exigir Cloudflare Turnstile em vez de bloquear.

#### ## Critérios de aceite

- [ ] Falhas vindas de outro IP não impedem o login correto do usuário.
- [ ] Força bruta de um mesmo IP continua limitada.
- [ ] Teste automatizado do cenário acima.

--- FIM ISSUE 2 ---

### --- ISSUE 3 ---

Título: [Segurança] Limite de 32 KB do corpo JSON pode ser contornado sem Content-Length

Labels: security, severidade: baixa

#### ## Descrição

`lerJson()` só recusa corpos grandes quando o cabeçalho `Content-Length` existe. Enviando o corpo em stream (chunked, sem `Content-Length`), `request.json()` lê tudo. O mesmo helper atende o `POST /api/contato`, que é público (a checagem de `Origin` não impede chamadas fora do navegador).

#### ## Evidência

```
`src/index.js:1413-1416`
```js
async function lerJson(request) {
  if (Number(request.headers.get('Content-Length') || 0) > 32000) return null;
  return request.json().catch(() => null);
}
},
},
```

Reprodução: `PATCH /api/eu` com 5 MB sem `Content-Length` respondeu `200 {"ok":true}`; com o cabeçalho, `400`.

Impacto

Consumo de CPU e memória do Worker com corpos grandes, contornando o teto pensado para a API.

Sugestão de correção

- Ler `request.body` com um contador de bytes e abortar ao passar de 32 KB, ou recusar escritas sem `Content-Length`.

Critérios de aceite

- [] Corpo de mais de 32 KB é recusado com e sem `Content-Length`.
- [] Teste automatizado com corpo em stream.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Endpoint do FormSubmit com e-mail real, sem captcha, e e-mail montado por terceiro

Labels: security, severidade: baixa

Descrição

O formulário do site usa o endereço real `contato@plannex.online` como endpoint do FormSubmit e envia `\_captcha=false`. Esse endereço funciona como credencial de envio: qualquer pessoa pode disparar e-mails para a caixa da Plannex direto no FormSubmit, sem passar pelo limite de envios da Central (5 a cada 10 minutos por IP). Além disso, nome e campos livres do cliente entram no assunto e no corpo de um e-mail montado pelo FormSubmit (template "table"), cujo escape de HTML está fora do controle do projeto.

Evidência

```
`public/index.html:393-396`
```html
<form action="https://formsubmit.co/contato@plannex.online" ...>
<input name="_template" type="hidden" value="table"/>
<input name="_captcha" type="hidden" value="false"/>
```
```

```
`public/assets/js/script.rev122.js:460`
```js
const FORM_SUBMIT_ENDPOINT = 'https://formsubmit.co/ajax/contato@plannex.online';
```

`public/assets/js/script.rev122.js:887`
```js
formData.set('_subject', `Plannex · ${central.protocolo} · Nova solicitação · ${serviceLabel} ·
${name}`);
```

## Impacto
Spam e e-mails falsos de "nova solicitação" na caixa da empresa (inclusive imitando pedidos reais) e dependência de um terceiro para escapar o conteúdo enviado pelo cliente.

## Sugestão de correção
- Trocar o endereço pelo alias aleatório que o FormSubmit oferece e ativar o captcha; ou
- Preferível: enviar o e-mail pelo próprio Worker, depois do limite por IP, com um serviço transacional e template com escape.

## Critérios de aceite
- [ ] O endereço real não aparece no HTML nem no JavaScript do site.
- [ ] Envios automatizados direto ao endpoint de e-mail são barrados (captcha ou limite no servidor).
- [ ] Dados do cliente no e-mail passam por escape controlado pelo projeto (se o envio sair do Worker).
--- FIM ISSUE 4 ---
```